

The Problem

The Problem: What Did We Solve?



Problem: The original Playlist Chaos app relied on rigid filtering using fixed energy thresholds and genre labels.



Users had to manually adjust sliders and understand technical attributes, which does not reflect how people think about music.

Filters

Energy Threshold

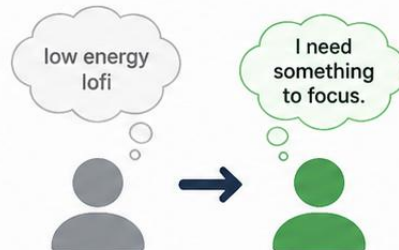
2 10

Genre

Lo-fi



Instead of
“low energy lofi,”
users think
“I need something
to focus.”



Additionally, the system
had bugs such as
incorrect search logic...

Search

lofi



Expected Results

Lo-fi Beats



Lofty Dreams



Lofi Study



Actual Results (Bug)

No results found



...and faulty statistics
returning 100% hype
ratio.

Mood Statistics



100%
Hype

0%
Chill

Bug:
Always shows
100% Hype



Our Goal: Create an intuitive, natural-language way to find the right music, while fixing critical bugs to ensure accuracy and reliability.



The Logic

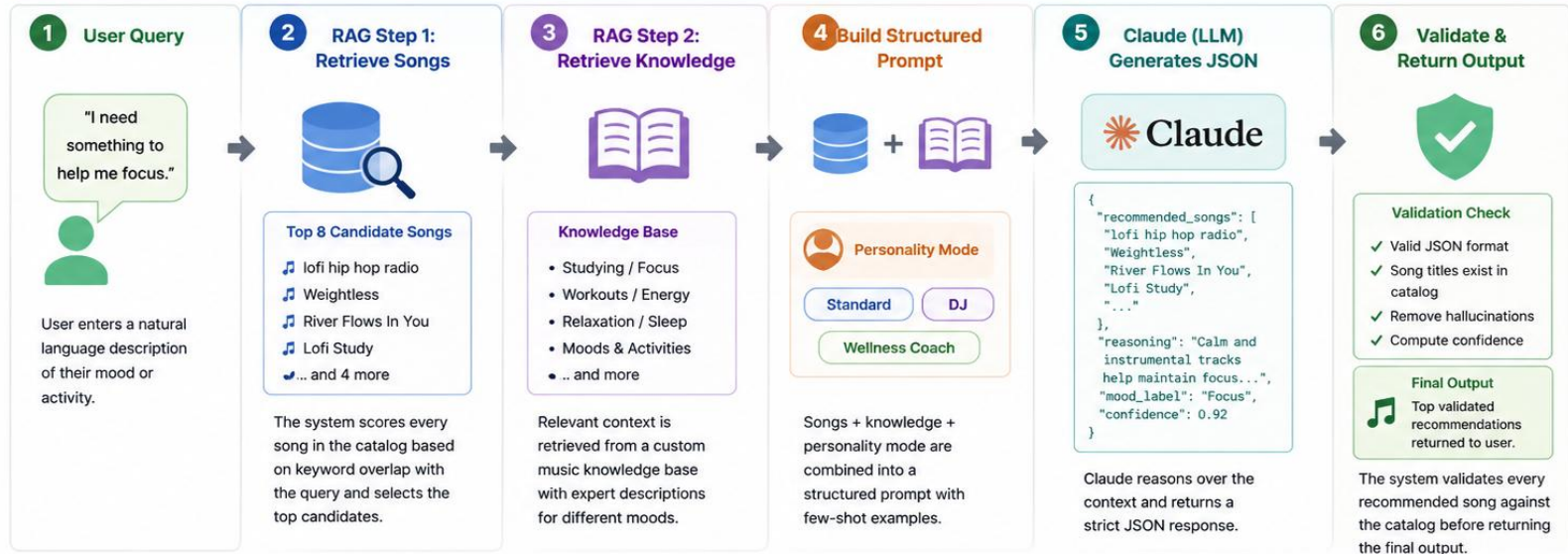


The Logic: How Does the AI Think?

Multi-Source RAG Pipeline with a Six-Step Agent Workflow



Goal: Understand the user's mood in natural language and return personalized song recommendations.



Example Structured Prompt (Simplified)

You are a music recommendation assistant.
Use the provided song catalog and knowledge to recommend songs that match the user's mood.
User Query: "I need something to help me focus."
Personality Mode: Standard

[Top 8 Candidate Songs]
[Relevant Knowledge: Studying / Focus]
[Example: (few-shot pair showing input → JSON output)]

Respond ONLY in valid JSON.



Example Validated Output

```
{
  "recommended_songs": ["lofi hip hop radio", "Weightless", "River Flows In You"],
  "reasoning": "These instrumental and ambient tracks help maintain concentration and reduce distractions.",
  "mood_label": "Focus",
  "confidence": 0.92
}
```

All songs validated and ready for user!



This pipeline enables the system to understand natural language, leverage multiple knowledge sources, and provide reliable, personalized music recommendations.

The Reliability



The Reliability: How Do We Know It Works?

Reliability is built into every layer of the system.

1 17 Unit Tests



A suite of 17 automated unit tests verifies core functions and ensures all identified bugs are properly fixed.

2 Strict JSON Formatting

```
{  
  "recommended_songs": [  
    "lofi hip hop radio",  
    "Weightless",  
    "River Flows In You"  
  ],  
  "confidence": 0.92  
}
```

All AI outputs follow strict JSON formatting to ensure consistency and ease of validation.

3 Catalog Validation & Filtering



Every recommended song is checked against the catalog. Invalid or hallucinated songs are removed before results are shown.

4 Evaluation Harness (Real Queries)



An evaluation harness runs 5 real-world queries end-to-end and reports pass/fail results with average confidence scores.

5 Logging & Audit Trail



```
[2025-05-19 10:15:23] QUERY:  
"I need music to focus"  
[2025-05-19 10:15:24] RESPONSE:  
{...}  
[2025-05-19 10:15:25] VALIDATED:  
3 songs returned  
[2025-05-19 10:15:25] CONFIDENCE:  
0.92
```

All AI interactions are logged with timestamps, inputs, outputs, and confidence scores for full transparency and debugging.



Result: The system delivers reliable, trustworthy, and transparent music recommendations.



Accurate



Safe



Transparent



Measurable

The Reflection



The Reflection: What Surprised Me?



Key Takeaway: Guardrails are essential not just for catching obvious failures, but for identifying subtle, near-miss errors that look correct to users.

1 Near-Correct > Completely Wrong

The model rarely produced completely wrong answers. Instead, it often generated near-correct outputs, such as slightly altered song titles or reformatted artist names.

User Query: "songs to help me focus"

Expected (In Catalog)		Model Output (Near-Correct)
Lo-fi hip hop radio	→	Lofi Hip-Hop Radio
River Flows In You	→	River Flows In You (Piano)
Weightless	→	Weightless – Marconi Union
Chillhop Essentials	→	Chillhop Essentials – Spring 2023



Why It Matters

Without validation, these near-misses would silently pass as correct results.

2 Prompt Design Strongly Shapes Output

Small changes in the few-shot examples and instructions dramatically changed the tone, style, and reasoning of the model's responses.



Same Query, Different Personalities

DJ Mode

"Let's keep the energy up! These tracks will keep the vibe alive! and the crowd moving."

Tone: Energetic, Hype, Club Vibes

Wellness Coach

"These calming tracks promote focus, reduce stress, and support mental clarity."

Tone: Calm, Supportive, Evidence-Based



Key Insight: The model is highly sensitive to tone examples — prompt design is a powerful tool for controlling behavior.



What I Learned

- Near-correct mistakes are harder to catch than total failures.
- Strong validation layers are critical for trustworthy AI systems.
- Prompt design is not just helpful — it enables true specialization (DJ, Coach, Friend, etc.).

Impact on My System



Catalog validation catches subtle errors



Reliable outputs users can trust



Personalized experiences through prompt modes